

The Maximum Subarray:-

Code:-

```
import java.io.*;
import java.util.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;
class Result {
    public static List<Integer> maxSubarray(List<Integer> arr) {
        int current = arr.get(0);
        int maxSubarray = arr.get(0);
        int maxSubsequence = arr.get(0);
        for (int i = 1; i < arr.size(); i++) {
            int num = arr.get(i);
            current = Math.max(num, current + num);
            maxSubarray = Math.max(maxSubarray, current);
            if (num > 0) {
                maxSubsequence += num;
            }
        }
        if (maxSubsequence == arr.get(0)) {
            maxSubsequence = Collections.max(arr);
        }

        List<Integer> result = new ArrayList<>();
        result.add(maxSubarray);
        result.add(maxSubsequence);

        return result;
    }
}

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader =
            new BufferedReader(new InputStreamReader(System.in));

        BufferedWriter bufferedWriter =
            new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));

        int t = Integer.parseInt(bufferedReader.readLine().trim());

        IntStream.range(0, t).forEach(tItr -> {
            try {
                int n = Integer.parseInt(bufferedReader.readLine().trim());

                List<Integer> arr = Stream.of(
                    bufferedReader.readLine().replaceAll("\\s+$", "").split(" ")
                )
                    .map(Integer::parseInt)
                    .collect(toList());
```

```

List<Integer> result = Result.maxSubarray(arr);

bufferedWriter.write(
    result.stream()
        .map(Object::toString)
        .collect(joining(" "))
        + "\n"
);

} catch (IOException ex) {
    throw new RuntimeException(ex);
}
});

bufferedReader.close();
bufferedWriter.close();
}
}

```

Output:-

✓ **Sample Test case 0**

✓ **Sample Test case 1**

Input (stdin)

[Download](#)

1	2
2	4
3	1 2 3 4
4	6
5	2 -1 2 3 4 -5

Your Output (stdout)

1	10 10
2	10 11

Expected Output

[Download](#)

Maximum Sum Circular Subarray:-

Code:-

```
class Solution {
    public int maxSubarraySumCircular(int[] nums) {
        int totalSum = 0;
        int maxKadane = nums[0], currentMax = 0;
        int minKadane = nums[0], currentMin = 0;
        for (int num : nums) {
            totalSum += num;
            currentMax = Math.max(num, currentMax + num);
            maxKadane = Math.max(maxKadane, currentMax);
            currentMin = Math.min(num, currentMin + num);
            minKadane = Math.min(minKadane, currentMin);
        }
        if (maxKadane < 0) {
            return maxKadane;
        }
        return Math.max(maxKadane, totalSum - minKadane);
    }
}
```

Output:-

The screenshot displays the LeetCode interface for the 'Maximum Sum Circular Subarray' problem. The submission is marked as 'Accepted' with 112/112 testcases passed. The user 'tameem_shaik_' submitted it on Sep 18, 2026, at 11:51. A promotional banner for 'School's in. Leet's on.' is visible. The performance metrics show a runtime of 6 ms (Beats 79.06%) and memory usage of 51.44 MB (Beats 9.03%). A bar chart illustrates the runtime distribution across different time intervals. The code editor shows the Java solution. The test result section for Case 1 shows the input array [1, -2, 3, -2], the output 3, and the expected result 3.

Runtime: 6 ms | Beats 79.06%

Memory: 51.44 MB | Beats 9.03%

Testcase: Case 1 (Accepted) | Runtime: 0 ms

Input: nums = [1, -2, 3, -2]

Output: 3

Expected: 3